Ensemble Methods: Boosting vs. Bagging

An Empirical Study Across Four Datasets

Emin Huseynli Ziyad Muradov Ismayil Yusifli

AI Academy – Machine Learning Final Project, Spring 2026

July 2026

Outline

Motivation & Central Question

Central Question

*Under what conditions does boosting outperform bagging, and vice versa — and **why**?*

Boosting (AdaBoost / SAMME)

- Re-weights hard examples sequentially
- Drives down bias
- Sensitive to label noise

Bagging (Random Forest)

- Bootstrap aggregation, trained independently
- Drives down variance
- More robust to noise

- All algorithms implemented from scratch in NumPy (no `sklearn` ensemble classes)
- Verified against `sklearn` on every dataset; 190 tests, 95% line coverage

Module 1 – Decision Tree (CART)

- Binary CART classifier: Gini impurity or entropy splitting
- Supports weighted samples (required by AdaBoost)
- `max_features` subsampling (required by Random Forest)
- Feature importances; matches sklearn within 2% on all four datasets

$$I_G(\mathrm{p}) = 1 - \sum_{c=1}^{C} p_c^2 \qquad \Delta I = I_{\text{parent}} - \frac{N_L}{N} I_L - \frac{N_R}{N} I_R$$

Module 2 – AdaBoost (SAMME)

Discrete SAMME using depth-1 decision stumps as weak learners. For $m = 1 \dots M$:

1. Fit stump h_m with sample weights $\mathrm{w}^{(m)}$
2. $\epsilon_m = \sum_i w_i^{(m)} \, 1[h_m(x_i) \neq y_i]$
3. $\alpha_m = \ln \dfrac{1 - \epsilon_m}{\epsilon_m} + \ln(K - 1)$
4. $w_i^{(m+1)} \propto w_i^{(m)} \exp\big(\alpha_m 1[h_m(x_i) \neq y_i]\big)$, renormalise

Final prediction: $\hat{y} = \arg\max_k \sum_{m=1}^{M} \alpha_m 1[h_m(x) = k]$

Bonus multi-class extensions: one-vs-rest wrapper and SAMME.R (real-valued).

Module 3 – Random Forest

- T trees, each on a bootstrap sample $\mathcal{D}_t$ (drawn with replacement)
- Each split considers only $\lfloor \sqrt{p} \rfloor$ random features
- Prediction by majority vote; `predict_proba` averages per-tree probabilities
- Out-of-bag (OOB) scoring: no-extra-cost validation proxy
- Parallel training via `multiprocessing.Pool`

Module 4 – Unsupervised Pipeline

PCA

- Eigendecomposition of the covariance matrix
- Scree plot, 2D projection

K-Means

- Lloyd's algorithm, k-means++ init
- 10 restarts per k, elbow method

DBSCAN

- Density-based, arbitrary shapes
- ε from k-distance knee

Datasets & Experimental Design

Dataset	Samples	Features	Task
Breast Cancer Wisconsin	569	30	Binary (63/37)
Credit Card Fraud	284,807	30	Binary, 0.17% fraud
MNIST (3 vs. 8 subset)	6,000	784	Binary, high-dim
Covertype (subset)	15,000	54	7-class

- Satisfies brief: severe imbalance ($\leq$1%) + high-dimensional ($>$20 features)
- Features standardised; SMOTE applied to Credit Card Fraud training fold only
- **7 experiments**, seeded (`random_state=42`), reproducible via `python src/experiments/run_all.py`

Exp 1 – Baseline Comparison

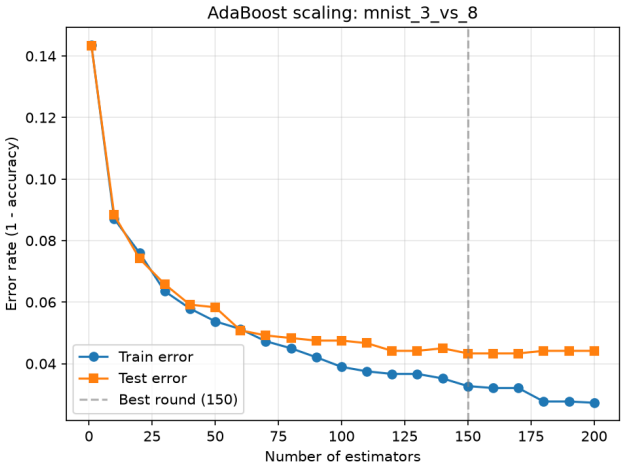
Unpruned tree, depth-1 stump, and sklearn's tree on identical 80/20 splits.

Dataset	Ours	sklearn	Diff.
Breast Cancer	0.9123	0.9123	0.0000
Credit Card Fraud	0.9991	0.9991	0.0000
MNIST (3 vs. 8)	0.9558	0.9583	0.0025
Covertype	0.7367	0.7370	0.0003

Max. gap 0.25 points (MNIST) – well within the 2% tolerance. A single stump already reaches 0.921 on Breast Cancer but only 0.639 on Covertype, motivating ensembling.

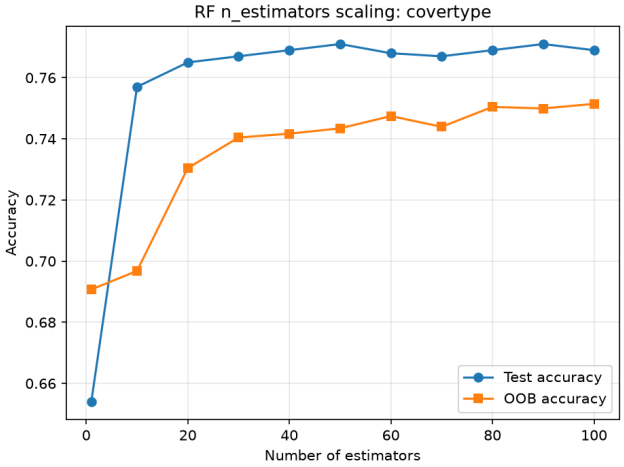
Exp 2 – AdaBoost Scaling

- $n\_estimators : 1 \rightarrow 200$, tracked via `staged_predict`
- Training error $\rightarrow 0$ within 30–40 rounds on all binary datasets
- Test error plateaus early: best round 10 (Breast Cancer), 150 (Credit Card, MNIST)
- No overfitting up to 200 rounds – margin-maximising behaviour



Exp 3 – Random Forest Scaling

- Vary *n*_estimators (1–100), `max_depth` (1–20)
- Accuracy rises sharply to ∼20 trees, then flat
- OOB accuracy tracks test accuracy within 1–2 points
- Deeper trees help most on Covertype / MNIST (7-class, 784-dim)



Exp 4 – Head-to-Head Comparison

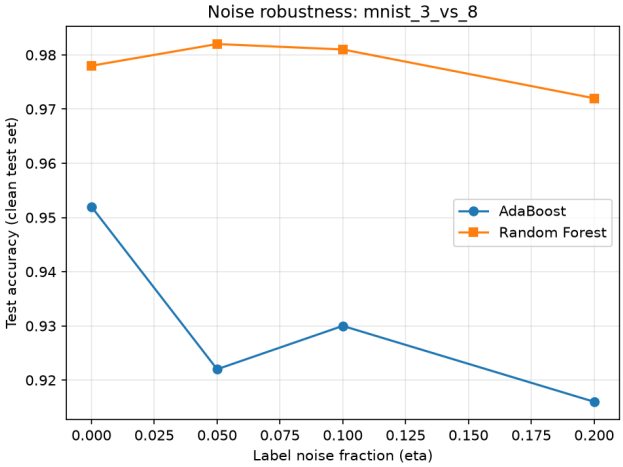
5-fold CV, $n\_estimators = 100$, mean $\pm$ std accuracy:

Dataset	Tree	AdaBoost	Our RF	sklearn RF
Breast Cancer	.910±.019	**.967±.018**	.951±.014	.956±.012
Credit Card	.992±.001	.995±.001	**.998±.001**	**.998±.001**
MNIST (3v8)	.952±.003	.950±.004	.979±.003	**.980±.002**
Covertype	.687±.010	.623±.014	.778±.014	**.784±.006**

AdaBoost wins on clean, balanced Breast Cancer; Random Forest wins (or matches sklearn) on all three harder datasets – widest gap on Covertype (15 points).

Exp 5 – Noise Robustness

- Flip $\eta \in \{5\%, 10\%, 20\%\}$ of training labels
- Breast Cancer, MNIST: AdaBoost degrades 3–6$\times$ faster than RF
- Credit Card, Covertype: pattern reverses, but gap $<$1.6 points either way

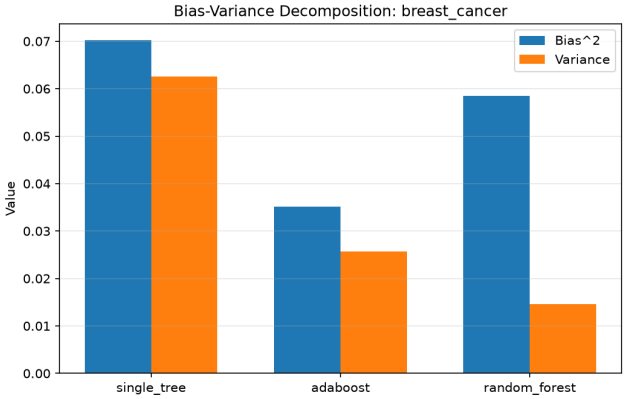


Exp 6 – Bias-Variance Decomposition

$B = 100$ bootstrap replicates (Breast Cancer):

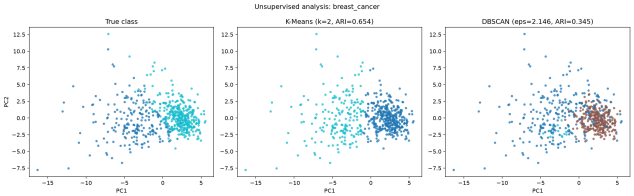
Model	Bias^2	Variance
Single Tree	0.0702	0.0626
AdaBoost	0.0351	0.0257
Random Forest	0.0585	0.0146

AdaBoost halves bias^2; Random Forest quarters variance – the textbook mechanism.



Exp 7 – Unsupervised Analysis

- Breast Cancer: 7 PCs @90% var., K-Means ARI 0.654 – classes well separated
- Credit Card Fraud: K-Means ARI ≈0 – fraud pattern invisible to distance clustering
- Covertype: DBSCAN (ARI 0.160) beats K-Means (0.044) – non-convex geometry



Cluster-label agreement tracks how separable each dataset's classes are – corroborates the supervised results.

Bonus – SAMME.R vs. One-vs-Rest AdaBoost

7-class Covertype, compute roughly matched ($\sim$100 stumps):

Variant	Stumps	Acc.	Log-loss	Time (s)
SAMME.R	100	0.674	2.650	4.62
OvR (naive)	700	**0.694**	**0.655**	26.96
OvR (compute-matched)	98	0.678	0.739	3.93

At matched compute, SAMME.R and OvR reach similar accuracy. Naive OvR (7$\times$ stumps) buys +2 accuracy points at 7$\times$ the training time – a poor trade-off. Log-loss result is a concrete discrepancy from the textbook expectation.

Key Findings

When does Boosting win?

Binary, reasonably separable classes, clean labels – lower bias^2, best accuracy on Breast Cancer.

When does Bagging win?

Severe class imbalance, high dimensionality, or >2 classes – lower variance, native multi-class handling, more consistent generalisation.

Unsupervised insight

High K-Means ARI (Breast Cancer) $\leftrightarrow$ best supervised accuracy; near-zero ARI (Credit Card, MNIST) $\leftrightarrow$ where RF's conservative variance profile earns its keep.

Conclusion

- Implemented Decision Tree, AdaBoost (SAMME), Random Forest, PCA, K-Means, DBSCAN from scratch (NumPy only)
- Verified against sklearn within 2% on four datasets
- 7 experiments + 2 bonus multi-class extensions (SAMME.R, matched-compute OvR)
- 190 tests, 95% line coverage
- **Limitations:** RF parallelism untested at scale; noise-robustness reversal on 2 datasets needs deeper study
- **Future work:** from-scratch Gradient Boosting Machine; extend bias-variance decomposition to all four datasets

Thank you – Questions?